

Тема урока: множественное наследование

1. Множественное наследование
2. Проблема ромбовидного наследования
3. Method Resolution Order (MRO)

Аннотация. Урок посвящен множественному наследованию в Python.

Множественное наследование

В предыдущих уроках мы познакомились с наследованием, или точнее с **одиночным наследованием**, при котором класс наследуется от одного другого класса. Помимо этого в Python реализовано **множественное наследование**, при котором класс наследуется от двух или более классов одновременно. В этом случае дочерний класс наследует все атрибуты и методы всех родительских классов.

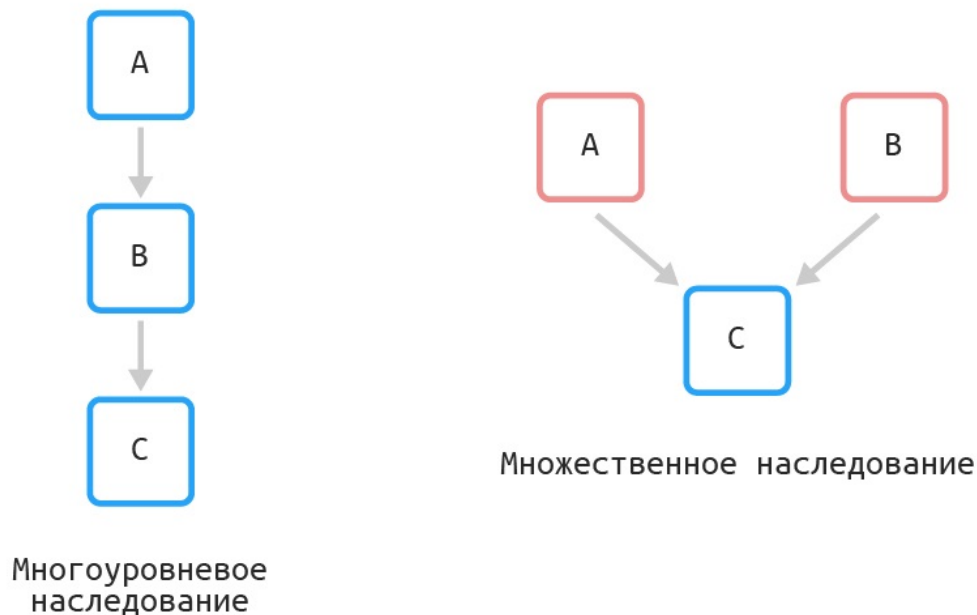
Синтаксис множественного наследования практически ничем не отличается от одиночного. Единственная разница в том, что при определении дочернего класса в скобках мы указываем не один родительский класс, а несколько через запятую:

```
class Parent1:                                # первый родительский класс
    pass

class Parent2:                                # второй родительский класс
    pass

class Child(Parent1, Parent2):                 # дочерний класс
    pass
```

Важно не путать множественное наследование с многоуровневым наследованием. При множественном наследовании класс определяется как наследник двух или более классов одновременно, при многоуровневом наследовании каждый класс цепочки определяется как наследник лишь одного класса:

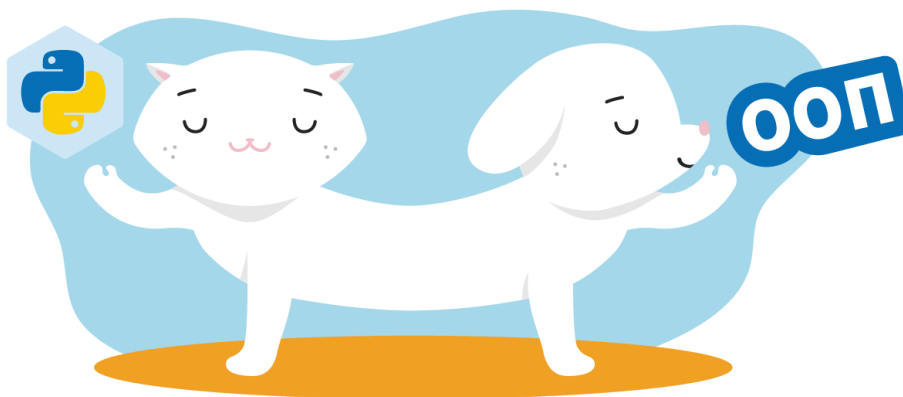


Рассмотрим иерархию из уже знакомых нам классов `Cat` и `Dog`, к которой добавим их наследника в лице класса `CatDog`, описывающего мультипликационного персонажа — котопса:

```
class Cat:
    def meow(self):
        return 'мяу'

class Dog:
    def bark(self):
        return 'гав'

class CatDog(Cat, Dog):
    pass
```



(https://stepik.org/media/attachments/lesson/841344/%D0%9B%D1%8E%D0%B1%D0%B8%D0%BC%D1%8B%D0%B9_%D0%BC%D1%83%D0%BB%D1%8C%D1%82%D0%B8%D0%BA_%D0%A2%D0%B8%D0%BC%D1%83%D1%80%D0%B0.mp4)

В классе `Cat` определен метод `meow()`, в классе `Dog` определен метод `bark()`, и класс `CatDog`, являясь их наследником, может пользоваться обоими этими методами.

Приведенный ниже код:

```
class Cat:
    def meow(self):
        return 'мяу'

class Dog:
    def bark(self):
        return 'гав'

class CatDog(Cat, Dog):
    pass

catdog = CatDog()

print(catdog.meow())
print(catdog.bark())
```

ВЫВОДИТ:

```
мяу
гав
```

Функционально множественное наследование повторяет одиночное наследование. Мы получаем возможность в дочернем классе пользоваться атрибутами и методами всех указанных родительских классов с последующим их переопределением или расширением. Однако множественное наследование, в отличие от одиночного, является потенциальным источником ошибок, которые могут возникнуть из-за наличия одинаковых имен атрибутов и методов в родительских классах.



При множественном наследовании класс может иметь неограниченное количество родительских классов.

В одиночном, а точнее в многоуровневом наследовании, иерархия классов представляет собой линейную структуру, в которой классы располагаются в том порядке, в котором наследуются друг от друга. И здесь ситуация, когда в нескольких родительских классах определен метод с одним и тем же именем, не является проблемой: текущим классом будет унаследован метод того родительского класса, который ближе к текущему в иерархии.

Приведенный ниже код:

```
class Animal:
    def sound(self):
        pass

class Cat(Animal):
    def sound(self):
        return 'мяу'

class Kitten(Cat):
    pass

kitten = Kitten()

print(kitten.sound())
```

ВЫВОДИТ:

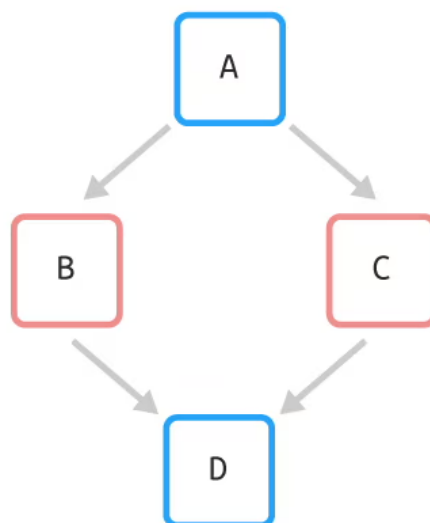
мяу

В случае множественного наследования иерархия классов представляет собой ветвистую структуру, а не линейную, и зачастую бывает не так просто определить, одноименный метод какого родительского класса будет унаследован текущим классом.

Проблема ромбовидного наследования

Проблема ромбовидного наследования — это типичная проблема при множественном наследовании. Она заключается в двусмысленности, возникающей, когда два класса **B** и **C** наследуются от класса **A**, а другой класс **D** наследуется от классов **B** и **C**.

Описанную иерархию можно изобразить следующим образом:



В обозначениях Python она будет иметь вид:

```
class A:
    pass

class B(A):
    pass

class C(A):
    pass

class D(B, C):
    pass
```

Указанная выше диаграмма иерархии классов `A`, `B`, `C` и `D` представляет собой ромб, именно поэтому неоднозначность, возникающую при подобном наследовании, называют проблемой ромбовидного наследования. Проблемой, потому что, например, если в классе `A` есть метод `method()`, который класс `B` или `C` переопределил (или они оба переопределили), то возникает вопрос, какую версию этого метода должен унаследовать класс `D`?

Рассмотрим одну из упомянутых ситуаций. Определим одноименный метод в классах `A`, `B` и `C` и проверим, какой из них будет унаследован классом `D`.

Приведенный ниже код:

```
class A:
    def method(self):
        print('Метод класса A')

class B(A):
    def method(self):
        print('Метод класса B')

class C(A):
    def method(self):
        print('Метод класса C')

class D(B, C):
    pass

d = D()

d.method()
```

ВЫВОДИТ:

Метод класса B

В данной ситуации класс `D` унаследовал метод класса `B`, и этот выбор не является случайным. В Python проблема ромбовидного наследования решается с помощью **MRO**.

Method Resolution Order (MRO)

В Python существует строгий порядок, в котором просматриваются классы во время поиска конкретного метода в иерархии классов при наследовании. Этот порядок определяется для каждого класса индивидуально в зависимости от его иерархии и называется **Method Resolution Order** или сокращенно **MRO**.

Для получения MRO класса достаточно воспользоваться методом `mro()`. Его результатом является список родителей класса, включая сам класс, расположенных ровно в том порядке, в котором Python будет производить поиск методов.

Приведенный ниже код:

```
class A:
    def method(self):
        print('Метод класса A')

class B(A):
    def method(self):
        print('Метод класса B')

class C(A):
    def method(self):
        print('Метод класса C')

class D(B, C):
    pass

print(D.mro())
```

ВЫВОДИТ:

```
[<class '__main__.D'>, <class '__main__.B'>, <class '__main__.C'>, <class '__main__.A'>, <class 'object'>]
```



MRO класса также можно получить с помощью атрибута класса `__mro__`.

Определив MRO класса, мы всегда можем точно сказать, какой из методов будет унаследован этим классом в том или ином случае, так как знаем, в каком порядке будут проверяться его родительские классы при поиске метода. Так, в нашей ромбовидной иерархии классов `A`, `B`, `C` и `D` мы видим, что в MRO класса `D` класс `B` находится перед классами `A` и `C`, следовательно, определенный во всех трех родительских классах метод `method()` будет унаследован именно из класса `B`.

Например, если метод `method()` будет определен только в классах `A` и `C`, то унаследованный метод будет принадлежать классу `C`, потому что в MRO класса `D` класс `C` находится перед классом `A`.

Приведенный ниже код:

```
class A:
    def method(self):
        print('Метод класса A')

class B(A):
    pass

class C(A):
    def method(self):
        print('Метод класса C')

class D(B, C):
    pass

d = D()

d.method()
```

ВЫВОДИТ:

Метод класса C

Аналогично, если метод `method()` будет определен только в классе `A`, то унаследованный метод будет принадлежать именно этому классу.

Приведенный ниже код:

```
class A:
    def method(self):
        print('Метод класса A')

class B(A):
    pass

class C(A):
    pass

class D(B, C):
    pass

d = D()

d.method()
```

ВЫВОДИТ:

Метод класса A

Наконец, если метод `method()` будет определен во всех родительских классах, включая сам класс `D`, то использоваться будет метод, реализованный именно в классе `D`.

Приведенный ниже код:

```
class A:
    def method(self):
        print('Метод класса A')

class B(A):
    def method(self):
        print('Метод класса B')

class C(A):
    def method(self):
        print('Метод класса C')

class D(B, C):
    def method(self):
        print('Метод класса D')

d = D()

d.method()
```

ВЫВОДИТ:

```
Метод класса D
```

MRO решает основную проблему множественного наследования — неоднозначность, возникающую при ромбовидном наследовании. Зная MRO класса, мы всегда можем определить какой метод будет унаследован этим классом, независимо от сложности его иерархии.



В MRO любого класса на первом месте всегда указывается исходный класс, а на последнем — класс `object`.

Простейшее построение MRO

В случае простейшего множественного наследования алгоритм построения MRO не вызывает трудностей, так как представляет собой поиск в глубину, слева направо, то есть сперва проверяется иерархия первого родительского класса, затем иерархия второго родительского класса, и так далее, с последующим исключением из полученной последовательности повторяющихся классов, кроме их последних включений.

Если мы попытаемся самостоятельно определить MRO класса `D` из нашей ромбовидной иерархии:

```
class A:
    pass

class B(A):
    pass

class C(A):
    pass

class D(B, C):
    pass
```

то сперва укажем класс `D`, затем иерархию первого родительского класса `B`, а после иерархию второго родительского класса `C`:

```
D, B, A, object, C, A, object
```

Затем исключим из полученной последовательности повторяющиеся классы, кроме их последних включений, и получим MRO класса `D`:

```
D, B, C, A, object
```

Данный способ не является решением в общем виде, он может служить лишь подсказкой при простейшем множественном наследовании, когда иерархии родительских классов представлены в виде линейных структур. В общем же случае алгоритм построения MRO имеет более сложный вид.



Алгоритм построения MRO называется C3-линеаризация. Ознакомиться с принципом его работы можно в официальной документации по [ссылке](https://www.python.org/download/releases/2.3/mro/)

(<https://www.python.org/download/releases/2.3/mro/>).

Невозможность построения MRO

Алгоритм, с помощью которого Python строит MRO класса, имеет ряд правил, в число которых входят следующие:

- **Правило наследования.** Если некоторый класс `D` является наследником классов `A`, `B`, `C` то при построении MRO эти классы должны располагаться в том порядке, в котором они были указаны в определении класса `D`, причем между ними могут располагаться другие классы, однако их исходный порядок должен остаться неизменным
- **Правило старшинства.** При построении MRO все классы иерархии должны располагаться в порядке старшинства, то есть ни один родительский класс не должен следовать перед своим дочерним классом.

Возможны ситуации, когда иерархия классов при множественном наследовании такова, что алгоритм при построении MRO не может соблюсти эти правила, в таких случаях построение MRO считается невозможным. Рассмотрим две ситуации множественного наследования, в которых н

соблюдается хотя бы одно из приведенных выше правил и MRO построить нельзя.

Ситуация 1. Построение MRO класса `C` при следующей иерархии классов:

```
class A:
    pass

class B(A):
    pass

class C(A, B):
    pass
```

является невозможным по той причине, что каково бы ни было MRO, не соблюдается либо правило наследования, либо правило старшинства:

- правило наследования нарушается при следующем построении MRO:

`C, B, A, object`

Так как классы `A` и `B` должны располагаться в том порядке, в котором были указаны в определении класса `C`

- правило старшинства нарушается при следующем построении MRO:

`C, A, B, object`

Так как ни один родительский класс не должен следовать перед своим дочерним классом, данном случае родительским классом является класс `A`, дочерним — класс `B`

Ситуация 2. Построение MRO класса `E` при следующей иерархии классов:

```
class A:
    pass

class B:
    pass

class C(A, B):
    pass

class D(B, A):
    pass

class E(C, D):
    pass
```

является невозможным по той причине, что по правилу наследования родители класса `C` должны быть расположены в порядке `A, B`, в то время как по тому же правилу родители класса `D` должны быть расположены в порядке `B, A`. В MRO класса `E` должны быть

соблюдены оба порядка, но этого сделать нельзя, так как они противоречат друг другу.

Примечания

Примечание 1. При недавнем рассмотрении функции `super()` было упомянуто, что она предоставляет доступ ко всем классам в иерархии наследования, а не к конкретному родительскому классу. При поиске необходимого метода в этой иерархии функция `super()` использует MRO.

Приведенный ниже код:

```
class A:
    pass

class B(A):
    pass

class C(A):
    def method(self):
        print('Метод класса C')

class D(B, C):
    def method(self):
        super().method()

d = D()

d.method()
```

ВЫВОДИТ:

```
Метод класса C
```

Примечание 2. Дополнительно о множественном наследовании можно почитать в официальной документации по ссылке (<https://docs.python.org/3/tutorial/classes.html#multiple-inheritance>).

Примечание 3. Обратите внимание, что даже самое простое множественное наследование является ромбовидным, так как все классы в Python являются наследниками класса `object` :

```
class A(object):
    pass

class B(object):
    pass

class C(A, B):
    pass
```

Примечание 4. При множественном наследовании атрибуты класса ведут себя так же, как и методы, то есть наследуются дочерним классом со всеми вытекающими особенностями.

Приведенный ниже код:

```
class Animal:
    en_name = 'animal'
    ru_name = 'животное'

class Cat(Animal):
    en_name = 'cat'
    ru_name = 'кот'

class Dog(Animal):
    en_name = 'dog'
    ru_name = 'собака'

class CatDog(Cat, Dog):
    pass

print(CatDog.en_name)
print(CatDog.ru_name)
```

ВЫВОДИТ:

```
cat
кот
```

